

# Projeto Interdisciplinar: Inteligência Artificial

## Entrega 2

### Grupo: Enrolados

Guilherme Barioni  
Iury Xavier da Silva Manguiera  
Lilian Mercedes Paye Conde  
Marcus Miranda Duque

### Relatório sobre a Prova de Conceito Funcional (PoC)

#### Introdução

O processo de contagem e triagem de alimentos em ações sociais ainda é realizado, em muitos casos, de forma manual, tornando a atividade lenta, cansativa e suscetível a erros humanos, como falhas de registro e duplicidade de contagem. Pensando nisso, o projeto **Visão Empática** propõe uma solução baseada em Visão Computacional e Inteligência Artificial capaz de automatizar a identificação e a contagem de alimentos em uma rampa de triagem. A solução utiliza modelos de detecção e classificação de objetos para reconhecer itens como arroz, feijão e outros mantimentos em tempo real.

Nesta Prova de Conceito (PoC), oferecemos uma experiência funcional e intuitiva. O sistema conta com um *dashboard* criado em Streamlit, que permite a criação de contas e equipes, a captura de imagens de alimentos, o gerenciamento dos dados de coleta e a visualização de métricas por equipe, validando o núcleo técnico da aplicação.



## Pipeline de Visão Computacional/IA

### Captura de imagem

Para realizar a captura dos alimentos, o nosso sistema se conecta com a webcam do usuário via navegador. O processamento do *stream* é feito utilizando a biblioteca OpenCV, que captura os quadros de vídeo em tempo real e os prepara para a inferência do modelo.

### Deteção, Classificação e Confiança

Para realizar a detecção e o recorte de ROI (Região de Interesse) dos pacotes na rampa, a aplicação utiliza a biblioteca Ultralytics para rodar um modelo da arquitetura YOLO (carregado a partir do arquivo `best.pt`). A cada quadro do vídeo processado, o modelo localiza as embalagens, retornando caixas delimitadoras (*bounding boxes*) e identificando suas respectivas classes (como Arroz, Feijão, Café, etc.).

Nativamente, a arquitetura YOLO acoplada ao sistema computa e avalia o nível de confiança (*confidence score*) para cada predição feita. A interface desenha sobreposições visuais (*overlays*) na tela com o nome dos itens e suas respectivas taxas de confiança, além de sumarizar as quantidades detectadas no frame em tempo real.



## Estratégia de Contagem

Na versão atual da PoC, a estratégia para evitar a duplicidade de contagem baseia-se em um controle assistido por *frames*. O vídeo executa o processamento continuamente, porém, os itens só são contabilizados de fato quando o operador utiliza a funcionalidade "Capturar Itens". Ao acionar esse gatilho, o sistema isola o último *frame* processado, armazena a sua imagem em Base64 e congela os dados das predições momentâneas em uma lista (salvando a classe do item, a quantidade, a confiança e marcando a data e hora exatas do evento). Dessa forma, um pacote na rampa não gera contagens múltiplas enquanto transita pelo campo de visão, pois o registro efetivo acontece de forma estanque e controlada, suprimindo a necessidade inicial de contagem única.

Dashboard

Detalhes do Grupo

Captura

Entrar

Grupo

Perfil

Deploy

### Captura com Detecção

☐ Ligar câmera

Capturar Itens **Ctrl+C**

#### Itens Capturados (Salvos)

| Item | Quantidade | Marca      | Peso | Data                | Imagem |
|------|------------|------------|------|---------------------|--------|
| Cafe | 1          | 3 Corações | 0.25 | 2026-05-02 23:15:48 |        |

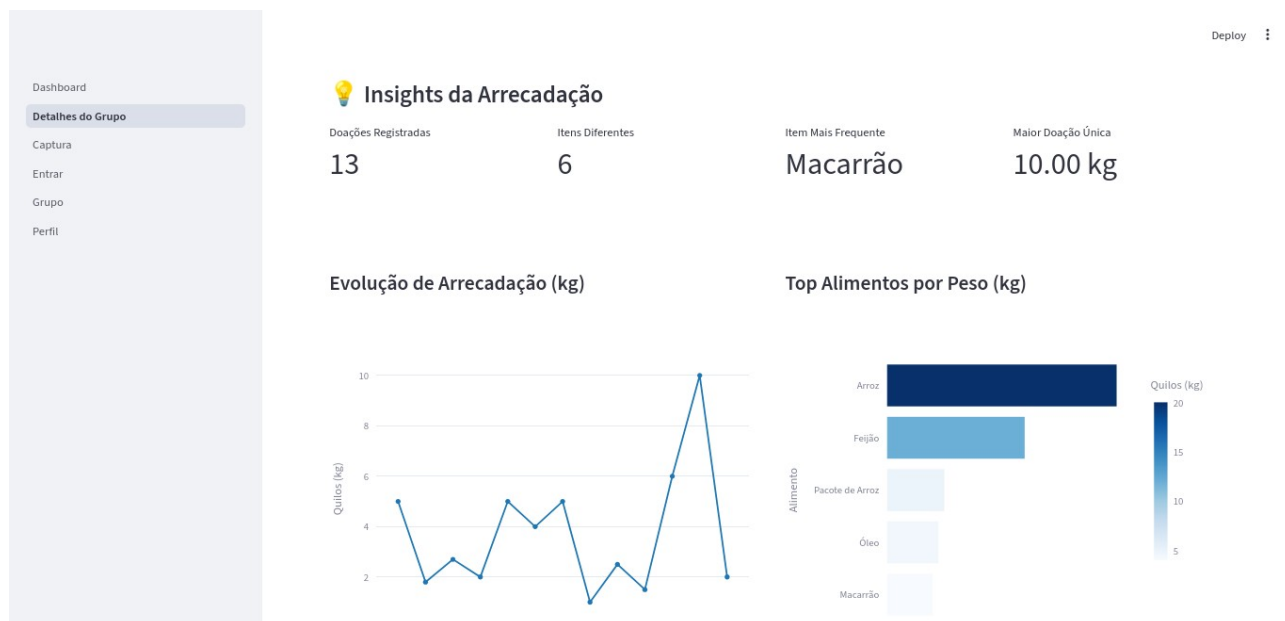
Enviar

Câmera desligada.

## Integração Mínima com Backend e Persistência

Quando o usuário termina o processo de captura, os dados coletados são enviados ao backend do projeto para processamento e consolidação. O backend, desenvolvido em Node.js, recebe as informações via a rota `/captura`. O código valida as informações, salva os *frames* de captura na pasta de *upload* e insere os registros em um banco de dados MySQL.

Essa persistência garante que cada evento de contagem seja devidamente registrado com um **timestamp (data e hora)** e vinculado à **equipe** responsável. Com esses dados integrados, o *dashboard* é capaz de renderizar um histórico de coletas com o auxílio visual de gráficos interativos, além de permitir a auditoria de coletas anteriores caso seja notada alguma inconsistência.



## Processo de treinamento do modelo

### Coleta dos dados

Para treinar o modelo, coletamos um conjunto inicial de 1.107 imagens diferentes dos alimentos mais arrecadados pelo projeto (arroz, feijão, açúcar, fubá, macarrão e óleo de soja). Essas imagens foram coletadas de diversos motores de busca como Google, Bing, Yahoo e DuckDuckGo.

As imagens foram separadas entre dados de treinamento (90%) e validação (10%) utilizando um script automatizado. Para a rotulação, utilizamos a ferramenta CVAT rodando localmente via Docker. Desenhamos manualmente as caixas delimitadoras para cada item e exportamos os dados rotulados no padrão YOLO Detection 1.0.

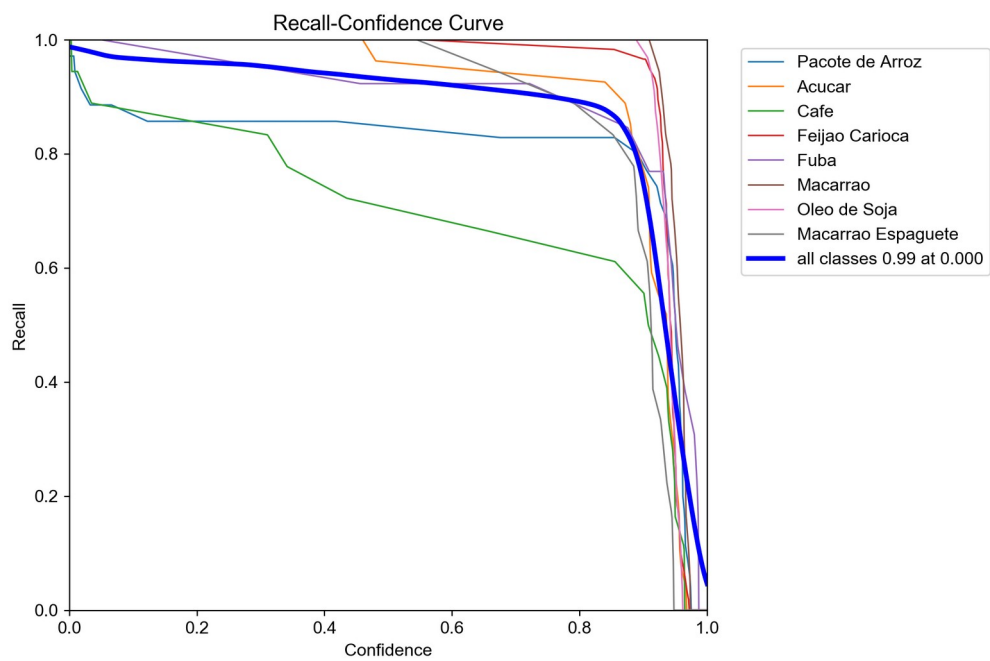
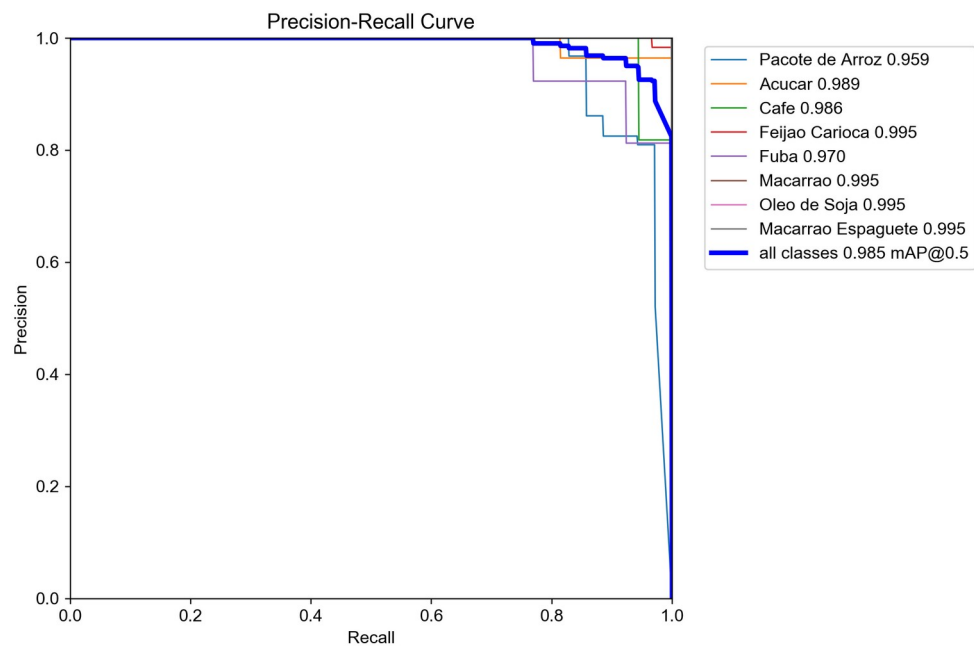
### Treinamento do modelo

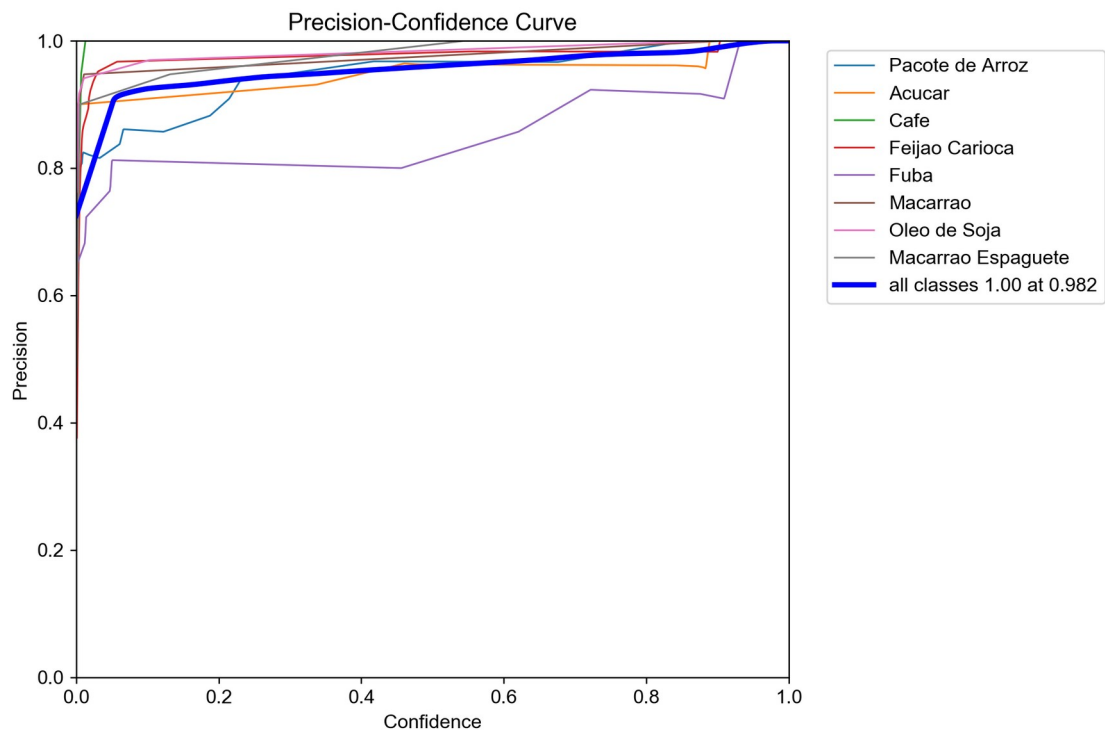
Com os dados rotulados em mãos, iniciamos o treinamento utilizando um *script* em Python. Com o intuito de garantir um modelo robusto, rápido e viável para tempo real, utilizamos a arquitetura **YOLOv8n** (Nano). A rede foi treinada por **100 épocas** (conforme registrado no histórico de perdas e métricas do pipeline).

### Resultados do treinamento

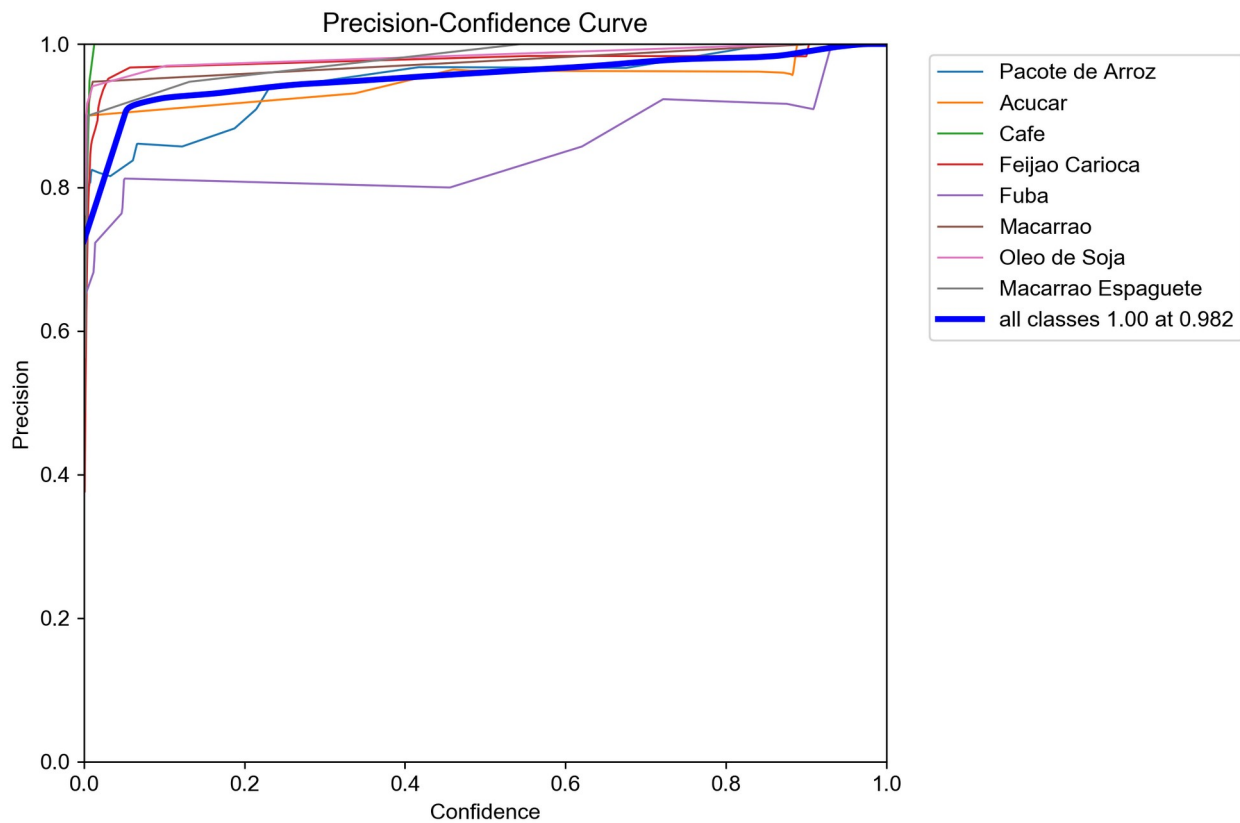
A partir da análise dos dados obtidos no término do treinamento (registrados no `results.csv` e nas curvas de performance geradas), o modelo atingiu uma estabilidade excelente na época 100, apresentando os seguintes resultados preliminares:

- **Precision (Precisão):** 94.4%
- **Recall (Revocação):** 95.7%
- **mAP@50:** 98.5%
- **mAP@50-95:** 93.6%





A curva **F1-Confidence** demonstra um valor máximo muito sólido de 0.95 (com confiança em 0.271), provando um ótimo equilíbrio entre Falsos Positivos e Falsos Negativos para o contexto do projeto.

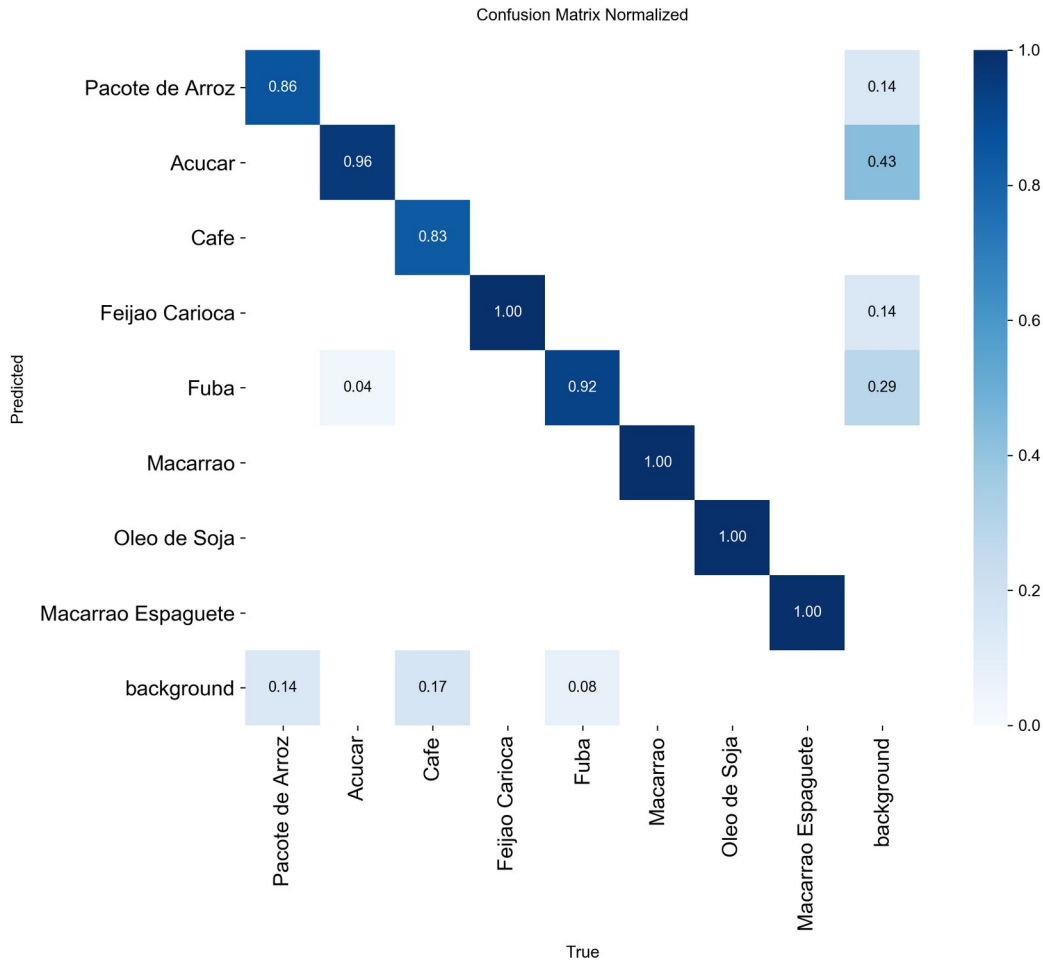


Matriz de Confusão e Análise de Erros:

Analisando as matrizes de confusão geradas, observa-se que o modelo é extremamente assertivo nas predições gerais. Categorias como *Feijão Carioca*, *Macarrão*, *Macarrão Espaguete* e *Óleo de Soja* apresentam índices de acerto próximos a 100% nas instâncias de validação.

No que tange à **Análise de Erros**, identificamos o seguinte comportamento preliminar:

- 1. **Falsos Positivos com Fundo (*Background*):** A maior parte dos erros do modelo envolve a confusão de elementos do ambiente (fundo) sendo previstos de forma errônea como *Açúcar*, *Pacote de Arroz* ou *Fubá*.
- 2. **Solução Proposta:** Como mitigação para as próximas versões, será necessário adicionar imagens negativas ao *dataset* (imagens do cenário vazio, sem produtos na rampa) para que o modelo aprenda a ignorar o fundo de forma mais eficiente.



Conclusão

Esta entrega cumpre de forma integral os requisitos de uma Prova de Conceito. A solução apresentada valida tecnicamente que é possível executar o *pipeline* mínimo exigido: captura de imagem, detecção das embalagens, classificação automatizada com registro de confiança e registro de eventos integrados a um banco de dados (com timestamps e separação por equipes).

O uso do gatilho de captura evitou o desafio de duplicidade nesta etapa inicial de forma simples e eficiente. Com um mAP50 de 98.5% já alcançado com o conjunto inicial de dados, a abordagem técnica escolhida se provou altamente funcional. O projeto encontra-se, portanto, preparado e maduro para evoluir para sua versão final, englobando a finalização da aplicação, aperfeiçoamento da modelagem do *background*, implementação de relatórios gerenciais e o *deploy* definitivo da infraestrutura.